PyTorch Fundamentals

Structured Notes

Training Loop, Loss Functions, Optimizers, Zero Gradient,
Learning Rate Effects, and Building Neural Networks with nn.Module

Contents

1 Three Essential Components of Training a Deep Learning Model **2**

2 Loss Function **2**
2.1 Loss Function Requirements . . . 2
2.2 Loss Function by Problem Type . . . 2

3 Optimizer **2**
3.1 Types of Gradient Descent . . . 3

4 The Training Loop **3**
4.1 Training Loop Implementation . . . 3

5 The Importance of Zero Gradient **4**
5.1 Effect on Loss Curve . . . 4
5.2 Training Loop Without Zero Gradient . . . 4

6 Understanding unsqueeze **4**

7 Learning Rate Effects **5**
7.1 Static vs Dynamic Learning Rate . . . 5
7.2 Learning Rate Too High: Exploding Gradients . . . 5
7.3 Finding the Optimal Learning Rate . . . 5

8 Troubleshooting Unstable Training **6**
8.1 Early Stopping . . . 6

9 From Scalar Linear Regression to Matrix Transformation **6**
9.1 nn.Linear Representation . . . 7
9.2 Matrix Multiplication Dimensions . . . 7

10 Building a Neural Network with nn.Module **7**
10.1 Defining a Neural Network Class . . . 7
10.2 Code Explanation . . . 8
10.3 How the Forward Method is Called . . . 8

11 Non-linear Activation Functions **8**

12 nn.Sequential: Simplified Network Definition **9**

13 Network Computation Summary **9**

14 The Black Box of Deep Learning **9**

15 Practical Considerations **10**
15.1 Hyperparameter Tuning . . . 10
15.2 Computational Constraints . . . 10

16 Summary of Key Concepts **10**

1 Three Essential Components of Training a Deep Learning Model

Training a deep learning model consists of three essential components:

1. **Model** — Contains weights and biases through which features are passed.
2. **Loss Function** — Calculates the error between model predictions and true target values.
3. **Optimizer** — Updates the model parameters to reduce the loss.

These three components interact through a structured procedure known as the **training loop**.

2 Loss Function

The **loss function** measures the difference between model predictions and true target values. Mathematically, loss is a function of prediction and target:

$$\text{Loss} = f(\hat{y}, y)$$

This function can be Mean Squared Error, Cross Entropy, or other loss functions depending on the problem type.

2.1 Loss Function Requirements

The loss function must satisfy three requirements:

1. It must be **differentiable** (for gradient computation).
2. It must return a **scalar** value.
3. It must **connect** the model output to the target.

The loss is the **starting point of back propagation**.

2.2 Loss Function by Problem Type

Problem Type	Loss Function	Output Type
Regression	Mean Squared Error (MSE)	Residuals
Binary Classification	Binary Cross Entropy	Logits
Multi-class Classification	Cross Entropy	Logits

3 Optimizer

Once gradients are computed, parameters must be updated. The **optimizer** updates the weights or parameters based on the computed gradients. For Stochastic Gradient Descent (SGD):

$$\theta = \theta - \eta \cdot \nabla_\theta L$$

where θ is the parameter, η is the learning rate, and $\nabla_\theta L$ is the gradient of the loss with respect to θ.

Critical Misconception: The optimizer does **NOT** compute gradients. Gradients are computed using **autograd**. The optimizer only **updates** the weights based on the gradients. The name "Stochastic Gradient Descent" causes confusion — students often think the optimizer calculates

gradients. This is incorrect.

3.1 Types of Gradient Descent

Type	Description
Batch Gradient Descent	Uses the complete dataset
Stochastic Gradient Descent (SGD)	Uses data in batches
Mini-batch Gradient Descent	Uses a random sample of the data

SGD is the most commonly used gradient descent variant.

4 The Training Loop

Every deep learning model follows the same training loop structure, whether it is ANN, CNN, or RNN. The training loop consists of five steps executed in strict order:

1. **Forward Pass:** Compute prediction by passing input through the model.
2. **Compute Loss:** Measure prediction error using the loss function.
3. **Backward Pass:** Compute gradients using autograd (`loss.backward()`).
4. **Optimizer Step:** Update the parameters (`optimizer.step()`).
5. **Zero Gradients:** Reset the gradients to prevent accumulation (`optimizer.zero_grad()`).

The order is critical. Any change in this order will affect model performance. The model will not run correctly if the order is changed.

4.1 Training Loop Implementation

Listing 1: Complete Training Loop with Zero Gradient

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

# Fix seed for reproducibility
torch.manual_seed(42)

# Create synthetic data
X = torch.linspace(0, 10, 100).unsqueeze(1)  # 100x1 tensor
Y = 3 * X**2 + 2 * X + torch.randn(100, 1)

# Define loss criterion
criterion = nn.MSELoss()
epochs = 80

# Create a simple linear model (single neuron)
model_A = nn.Linear(1, 1)  # W: 1x1, bias: 1

# Define optimizer
optimizer_A = optim.SGD(model_A.parameters(), lr=0.01)

# Training loop WITH zero grad
losses = []
for epoch in range(epochs):
    prediction = model_A(X)           # Forward pass
```

```
    loss = criterion(prediction, Y)    # Compute loss
    loss.backward()                    # Backward pass
    optimizer_A.step()                 # Update parameters
    optimizer_A.zero_grad()            # Reset gradients
    losses.append(loss.item())
```

5 The Importance of Zero Gradient

PyTorch **accumulates gradients by default**. If the gradients are not reset to zero after every epoch, the new gradient computed in the next epoch will be **added** to the previous gradient instead of replacing it.

Example of gradient accumulation:

- After epoch 1: gradient $= G_1$
- After epoch 2 (without reset): gradient $= G_1 + G_2$
- After epoch 3 (without reset): gradient $= G_1 + G_2 + G_3$
- This continues: $G_1 + G_2 + G_3 + \cdots + G_{80}$

The gradients do not overwrite — they keep adding. This causes the loss to become unstable and never converge.

5.1 Effect on Loss Curve

With zero gradient: The loss curve is smooth and converges to a minimum value. The loss decreases steadily over epochs.

Without zero gradient: The loss curve shows significant fluctuation and noise. The loss does not converge because accumulated gradients cause incorrect weight updates. It will never converge.

Placement of zero_grad(): Whether `optimizer.zero_grad()` is placed at the beginning or end of the loop does not affect model accuracy. The key requirement is that after every epoch is completed, the gradient must be reset to zero. Placing it at the end is preferred because it clears the gradient memory (RAM) after the loop finishes, freeing up memory.

5.2 Training Loop Without Zero Gradient

Listing 2: Training Loop WITHOUT Zero Gradient (Incorrect)

```
# Training loop WITHOUT zero grad
losses_no_zero = []
model_B = nn.Linear(1, 1)
optimizer_B = optim.SGD(model_B.parameters(), lr=0.01)

for epoch in range(epochs):
    prediction = model_B(X)
    loss = criterion(prediction, Y)
    loss.backward()
    optimizer_B.step()
    # Missing: optimizer_B.zero_grad()
    losses_no_zero.append(loss.item())
```

6 Understanding unsqueeze

`unsqueeze(1)` adds a dimension to the data tensor. PyTorch requires data to be in the correct shape for matrix multiplication.

- Without `unsqueeze`: `torch.linspace(0, 10, 100)` produces a 1D tensor of size `[100]`.
- With `unsqueeze(1)`: The tensor becomes shape `[100, 1]` — a 2D tensor suitable for matrix multiplication.

This converts a vector into a 2D tensor. If this is not used, a shape mismatch error will occur because PyTorch works only if the shape is correct.

7 Learning Rate Effects

The **learning rate** controls the step size during parameter updates. It directly affects whether the model converges and how quickly.

7.1 Static vs Dynamic Learning Rate

Static learning rate: The same learning rate value is used for all epochs. For example, $\eta = 0.01$ for all 80 epochs.

Dynamic learning rate: The learning rate changes during training. After some epochs, if the loss is not decreasing or converging, the learning rate is adjusted. Common strategies:

- Use an exponential decay function (e.g., e^{-x} curve)
- Monitor validation/training loss for 5–10 epochs; if constant, decrease the learning rate
- Repeat the process until convergence is achieved

7.2 Learning Rate Too High: Exploding Gradients

Experiment results with different learning rates:

Learning Rate	Behavior
$\eta = 0.01$	Converges, loss stabilizes around 1.7
$\eta = 0.02$	Converges, loss decreased to 1.1 (improvement)
$\eta = 0.05$	**Explodes** — weights go to infinity
$\eta = 0.1$	**Explodes** — gradient explosion, NaN/Inf values

When the learning rate is too high, the weights overshoot the optimal values, causing **exploding gradients**. The weight values go to infinity.

7.3 Finding the Optimal Learning Rate

Manual approach:

1. Start with a learning rate and observe the loss curve.
2. If loss explodes, reduce the learning rate.
3. Find the range where convergence occurs (e.g., 0.01 to 0.02).
4. Within that range, use smaller steps (e.g., 0.013, 0.015) to find the best value.

Cross-validation approach: Create a set of learning rate values and apply cross-validation. This is the best method but is computationally expensive for deep learning — it can take days or weeks

on large datasets.

Practical note: First find the range manually, then use cross-validation within that range to find the optimal value.

8 Troubleshooting Unstable Training

If training becomes unstable, check the following components in order:

1. Is `loss.backward()` being called?
2. Is `optimizer.step()` called **after** `loss.backward()`?
3. Is `zero_grad()` set?
4. Is the learning rate appropriate?

Common causes of loss not converging:

- **Learning rate too high:** Causes exploding gradients or divergence.
- **Missing zero_grad():** Gradient accumulation causes noisy, unstable loss.
- **Incorrect training loop order:** Steps must follow the exact sequence.
- **Data quality issues:** The data itself may not be suitable for training.
- **Scaling/normalization issues:** Improper data scaling can affect convergence.
- **Batch normalization or dropout issues:** These regularization techniques can cause instability if misconfigured.

8.1 Early Stopping

Early stopping is used when the loss has converged and oscillates between very small values (e.g., 1.71, 1.70, 1.72). If training is set for 500 epochs but convergence is reached at 200 epochs, early stopping saves time and prevents overfitting. The criterion is typically: if the loss does not improve for 5–10 consecutive epochs, stop training.

9 From Scalar Linear Regression to Matrix Transformation

Scalar form: $y = wx + b$ (single weight, single input)

Matrix form: For batch input with multiple features:

$$Y = XW^T + b$$

where:

- $X \in \mathbb{R}^{N \times D}$ — N is the batch size, D is the number of input features
- $W \in \mathbb{R}^{K \times D}$ — K is the number of output features, D is the number of input features
- $b \in \mathbb{R}^K$ — bias vector with K elements
- $Y \in \mathbb{R}^{N \times K}$ — output with N samples and K output features

This is the fundamental building block of all neural networks. Linear algebra matrix transformation is the core operation.

9.1 nn.Linear Representation

`nn.Linear(D, K)` represents:

- A learnable weight matrix W with dimension $K \times D$
- A learnable bias b with dimension K
- The complete transformation $Y = XW^T + b$ in a single line of code

Example: `nn.Linear(1, 8)` creates:

- Weight matrix: 8×1
- Bias: 8
- Input: shape $[N, 1] \rightarrow$ Output: shape $[N, 8]$

D (input features) is determined by the data. K (output features/neurons) is chosen by the user. K defines the number of neurons (depth), while the number of layers defines the width of the network.

9.2 Matrix Multiplication Dimensions

For an input X with shape $[N, D]$ and weight W^T with shape $[D, K]$:

$$[N \times D] \cdot [D \times K] = [N \times K]$$

Concrete example:

- Input: $X \in \mathbb{R}^{100 \times 1}$ (100 samples, 1 feature)
- Layer 1 weight: $W_1^T \in \mathbb{R}^{1 \times 8}$
- After Layer 1: $[100 \times 1] \cdot [1 \times 8] = [100 \times 8]$
- ReLU activation: $[100 \times 8]$ (dimension unchanged)
- Layer 2 weight: $W_2^T \in \mathbb{R}^{8 \times 1}$
- After Layer 2: $[100 \times 8] \cdot [8 \times 1] = [100 \times 1]$

If parameters are passed incorrectly, a **shape mismatch error** will occur.

10 Building a Neural Network with nn.Module

`nn.Module` provides a structured way to define neural networks with:

- **Learnable matrices:** Automatic parameter registration.
- **Clean forward computation:** Structured definition of how data flows through the network.
- **Scalability:** Instead of manually defining weight and bias for every neuron, layers are defined as matrix transformations, making deep networks manageable.

If billions of parameters had to be defined individually, it would take an impractical amount of time. Instead, matrix transformations handle this automatically.

10.1 Defining a Neural Network Class

Listing 3: Neural Network using nn.Module

```
1 import torch.nn as nn
```

```
2
3  class SimpleNN(nn.Module):
4      def __init__(self):
5          super(SimpleNN, self).__init__()
6          # First linear transformation: input 1, output 8
7          self.layer1 = nn.Linear(1, 8)   # W: 8x1, b: 8
8          # Non-linear activation
9          self.relu = nn.ReLU()
10         # Output layer: input 8, output 1
11         self.layer2 = nn.Linear(8, 1)   # W: 1x8, b: 1
12
13     def forward(self, x):
14         x = self.layer1(x)   # Linear transformation
15         x = self.relu(x)     # Non-linear activation
16         x = self.layer2(x)   # Output transformation
17         return x
18
19 model = SimpleNN()
```

10.2 Code Explanation

Class structure:

- `class SimpleNN(nn.Module)` — Inherits from `nn.Module`.
- `__init__` — Constructor that defines the layers.
- `super().`$_{init_{()}}$$---Callstheparentclassconstructor(requiredfor$`nn.Module`$).$`self`$---Referencetotheobject(pointertotheinstance).$
- `forward(self, x)` — Defines how data passes through the network.

Layer definitions:

- `self.layer1 = nn.Linear(1, 8)` — First layer with 8 neurons, weight shape 8×1.
- `self.relu = nn.ReLU()` — Rectified Linear Unit activation function.
- `self.layer2 = nn.Linear(8, 1)` — Output layer with 1 neuron, weight shape 1×8.

10.3 How the Forward Method is Called

When `model(X)` is called, PyTorch automatically invokes the `forward` method. This is because `nn.Module` has a built-in `forward` method that is overridden in the child class. This is an example of **polymorphism** (method overriding) from object-oriented programming. The forward function is defined in the parent class `nn.Module`, but overwritten in the child class with custom layer logic.

11 Non-linear Activation Functions

ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is used to introduce **non-linearity** into the network. It does not change the tensor dimensions — it only modifies the values:

- Negative values are set to 0.

- Positive values remain unchanged.

ReLU determines which features are important (non-zero) and which to suppress (zero).

Without activation functions: Stacking linear layers without non-linear activation between them reduces to a **single linear layer**. The network would be equivalent to linear regression, unable to capture non-linear patterns. If there is no non-linearity, deep learning is not even needed — a simple linear regression would suffice.

With activation functions: The power of neural networks comes from **repeated matrix transformation with non-linear activation between layers**. This is what enables neural networks to capture deep, hidden patterns that normal ML functions cannot identify.

After every hidden layer, an activation function must be defined. The choice of activation function is a hyperparameter that can be tuned.

12 nn.Sequential: Simplified Network Definition

`nn.Sequential` provides a simpler way to define neural networks by specifying layers in sequence. It automatically chains the layers together.

```
model = nn.Sequential(
    nn.Linear(1, 8),
    nn.ReLU(),
    nn.Linear(8, 1)
)
```

This is equivalent to defining a class with `nn.Module` but is more concise. `nn.Sequential` adds hidden layers between the input and output layers in the order specified.

13 Network Computation Summary

The complete network computes:

$$Y = W_2^T \cdot \text{ReLU}(W_1^T \cdot X + b_1) + b_2$$

Step-by-step:

1. **Matrix multiplication:** $H = X \cdot W_1^T + b_1$ (linear transformation)
2. **Non-linearity:** $A = \text{ReLU}(H)$ (activation)
3. **Matrix multiplication:** $Y = A \cdot W_2^T + b_2$ (output transformation)

This pattern of matrix multiplication → non-linearity → matrix multiplication repeats throughout all neural network architectures (ANN, CNN, RNN). Additional operations like batch normalization may be added between layers. CNN uses convolution layers, but fundamentally it is matrix multiplication with a small filter matrix.

14 The Black Box of Deep Learning

The term "black box" in deep learning refers to the inability to **visualize** what happens inside the network beyond three dimensions. Up to three dimensions, matrix multiplication can be visualized. Beyond that, visualization is lost.

Key clarification:

- The "black box" is about **interpretation and visualization**, not about understanding the mathematics.
- The mathematics is known — it is matrix multiplication.
- Saying "we don't know how it works" is incorrect. The correct statement is "we cannot visualize how it works beyond three dimensions, but we know the mathematics."

15 Practical Considerations

15.1 Hyperparameter Tuning

Key hyperparameters in neural networks:

- **Learning rate:** Step size for parameter updates.
- **Number of epochs:** How many times the model sees the full dataset.
- **Number of neurons per layer (K):** Determines the width of the network.
- **Number of layers:** Determines the depth of the network.
- **Activation function:** Type of non-linearity (ReLU, sigmoid, tanh, etc.).
- **Batch size:** Number of samples processed before a weight update.

Cross-validation can be used to tune hyperparameters, including learning rate. In frameworks like Keras, there is a module called parameter tuner. However, cross-validation for deep learning is expensive and can take days (e.g., 5–6 days for large datasets).

15.2 Computational Constraints

Training a model on large datasets (e.g., 200 GB) can take 5–6 hours per training run. If the learning rate needs to be changed, the entire training must be repeated. This is why hyperparameter tuning can take weeks or months in practice. Cross-validation is expensive for deep learning, but results are solid. Companies can afford the computational resources, while individual practitioners may face constraints.

16 Summary of Key Concepts

Concept	Key Point
Training Loop	Forward pass → Loss → Backward → Step → Zero grad
Loss Function	Must be differentiable, scalar, connects output to target
Optimizer (SGD)	Updates weights only; does NOT compute gradients
Zero Gradient	Prevents gradient accumulation; essential for convergence
Learning Rate	Too high → exploding gradients; too low → slow convergence
Early Stopping	Stop training when loss plateaus to save time
`nn.Linear(D, K)`	Linear transformation: $Y = XW^T + b$
`nn.Module`	Base class for defining neural networks
`nn.Sequential`	Sequential layer stacking for simpler definitions
ReLU	$\max(0, x)$; introduces non-linearity
`unsqueeze()`	Adds dimension for correct matrix multiplication shape
Dynamic Learning Rate	Adjust learning rate during training for better convergence